

Knapsack Problem using a Greedy Algorithm

First, what is the Knapsack Problem?

Imagine you have:

- A bag (called knapsack) that can carry a limited weight.
- A bunch of items. Each item has:
- A weight.
- A value (how much it's worth).

Goal:

Pick items to put in your bag so that:

The total value is as large as possible.

You don't exceed the bag's weight limit.

Now: using a Greedy Algorithm for Knapsack

Types

There are two types of knapsack:

0/1 Knapsack: You either take the whole item or leave it (can't break it).

Fractional Knapsack: You can take fractions of an item (like half of it).

☐ Greedy works perfectly only for Fractional Knapsack.

☐ For 0/1 Knapsack, greedy does not always give the best answer.

How the Greedy Algorithm Works (for Fractional Knapsack):

Idea:

Pick items with the highest value per weight first.

This is also called "value density" = $(\text{value} \div \text{weight})$.

Steps:

- Calculate value per weight for each item.
- Sort all items by value per weight (highest first).
- Start putting items into the bag:
- If the whole item fits, take it all.
- If it doesn't fit, take the fraction that fits.

Example

Suppose:

Bag capacity = 50 kg

Items:

Item A: value = 60, weight = 10

Item B: value = 100, weight = 20

Item C: value = 120, weight = 30

Step 1: Calculate value/weight for each

Item A: $60/10 = 6$

Item B: $100/20 = 5$

Item C: $120/30 = 4$

Step 2: Sort items by value/weight (highest first)

Item A (6)

Item B (5)

Item C (4)

Step 3: Fill the bag

Take all of Item A (10 kg). Remaining capacity = $50 - 10 = 40$ kg

Take all of Item B (20 kg). Remaining capacity = $40 - 20 = 20$ kg

Take $20/30 = 2/3$ of Item C.

Value from Item C = $2/3 \times 120 = 80$.

Total value:

Item A: 60

Item B: 100

$2/3$ of Item C: 80

Total = $60 + 100 + 80 = 240$

Quick Code (Python-style)

```
class Item:
```

```
    def __init__(self, value, weight):
```

```
        self.value = value
```

```
        self.weight = weight
```

```
        self.ratio = value / weight
```

```
def fractional_knapsack(items, capacity):
```

```
    items.sort(key=lambda x: x.ratio, reverse=True) # Sort by value/weight
```

```
    total_value = 0.0
```

```
    for item in items:
```

```

if capacity >= item.weight:

    capacity -= item.weight

    total_value += item.value

else:

    total_value += item.ratio * capacity

    break

return total_value

```

Example

```

items = [Item(60, 10), Item(100, 20), Item(120, 30)]

capacity = 50

print(f"Maximum value in Knapsack = {fractional_knapsack(items, capacity)}")

```

Summary Table

Step	Action	Remaining Capacity	Total Value
Take all A	10 kg taken	40 kg	60
Take all B	20 kg taken	20 kg	160
Take 2/3 of C	20 kg taken	0 kg	240